Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Übung 05: VHDL - sequentielle Schaltung

M.Sc. E. Trumann, M.Sc. J. Homann, Prof. Dr.-Ing. G. Payá Vayá,
21. November 2025

Register und Latches

Das Signal *b* soll den Wert von *a* annehmen, wenn *sel* gesetzt ist, ansonsten auf '1' gesetzt werden. Welche Schaltung wird durch folgende Beschreibung gebaut?

```
architecture rtl of mux is
begin
        process(sel, a)
        begin

                if sel = '1' then
                        b <= a;
                end if;
        end process;
end architecture rtl;
```

Register und Latches

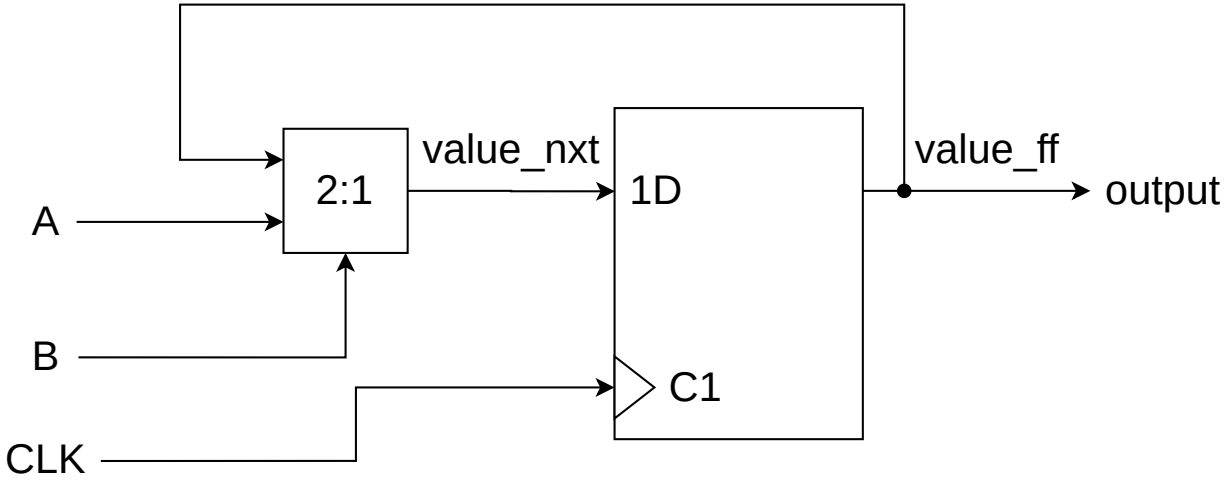
Das Signal *b* soll den Wert von *a* annehmen, wenn *sel* gesetzt ist, ansonsten auf '1' gesetzt werden. Welche Schaltung wird durch folgende Beschreibung gebaut?

```
architecture rtl of mux is
begin
        process(sel, a)
        begin

                if sel = '1' then
                        b <= a;
                end if;
        end process;
end architecture rtl;
```

Wie könnte man den Code korrigieren?

Register und Latches

Beschreiben Sie folgende Schaltung in VHDL:



Register und Latches

```
entity register is
port(
CLK    : in std_ulogic;
A      : in std_ulogic;
B      : in std_ulogic;
output : out std_ulogic
);
end register;
architecture RTL of register is




begin
```

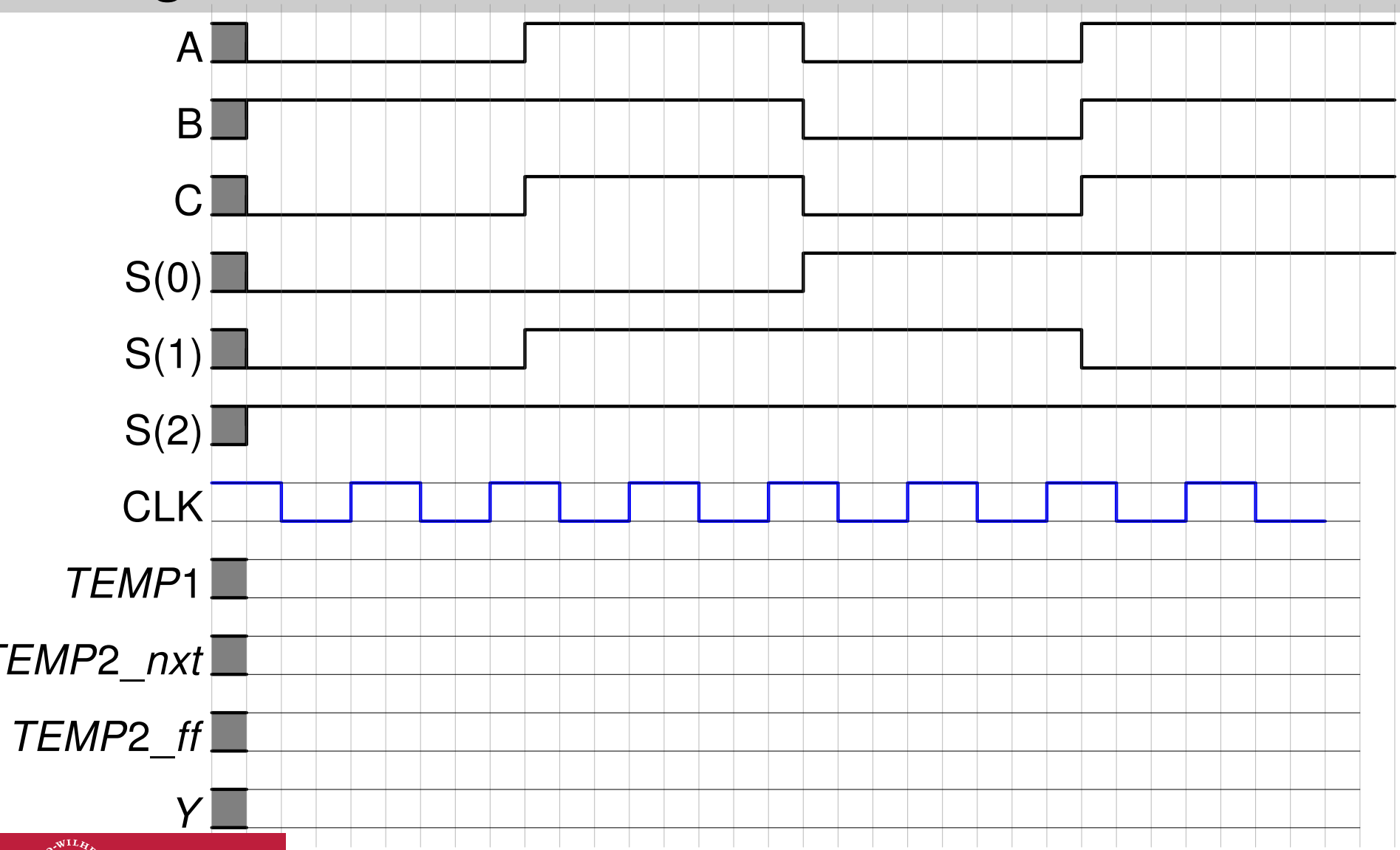
Iterative Instanziierung: Beispiel

```
seq: process(clk)
begin
        if rising_edge(clk) then
                TEMP2_ff <= TEMP2_nxt;
        end if;
end process;

MUX1: process (A, TEMP1, S(0))
begin
        TEMP2_nxt <= (A and not S(0)) or (TEMP1 and S(0));
end process;

MUX2: process (B, C, S(1))
begin
        TEMP1 <= (B and not S(1)) or (C and S(1));
end process;

Y <= TEMP2_ff xor S(2);
```

Register und Latches

Welche Schaltung wird mit dieser Beschreibung erzeugt?

Register und Latches



Zähler

Es soll ein 3-Bit-Zähler beschrieben werden. Der Wert des Zählers soll in einem Register gespeichert werden und jeden Takt um 1 inkrementieren und bei 7 stoppen bzw. den Wert halten. Falls am Eingang reset_cnt während einer steigenden Taktflanke eine 1 anliegt, soll der Zähler den Wert 0 annehmen.

Zähler

cnt_ff_2	cnt_ff_1	cnt_ff_0		cnt_nxt_2	cnt_nxt_1	cnt_nxt_0
0	0	0				
0	0	1				
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				

Zähler

Implementieren Sie nun den Zähler. Verwenden sie eine case-Struktur, um den Zähler zu implementieren. Benutzen Sie dabei einen sequentiellen Prozess für die Beschreibung des Registers und einen kombinatorischen Prozess für die Berechnung des nächsten Werts.

Aus der Vorlesung: `generate`-Ausdrücke

- Die sog. `generate`-Ausdrücke erhöhen die Lesbarkeit und die Wiederverwendbarkeit einer VHDL-Modulbeschreibung
  - Bei hoher HW-Redundanz, z.B. viele Instanzen eines Moduls, kann ein `for-generate` Konstrukt die Lesbarkeit erhöhen
  - Bei hochparametrisierten Beschreibungen, z.B. wenn die Top-Ebene einer Systembeschreibung mehrere Implementierungsalternativen beinhalten soll, ist ein `if-generate` Konstrukt unerlässlich

```
identifier_1 : for Parameter in Wert_1 to Wert_2 generate
identifier_2 : if Bedingung generate
Komponenteninstanziierung
end generate identifier_2;
end generate identifier_1;
```

→ Der `Parameter` im `generate`-Schema ist eine statische Größe, die hier implizit deklariert wird, nur innerhalb der `generate`-Anweisung verfügbar ist und nicht verändert werden darf.

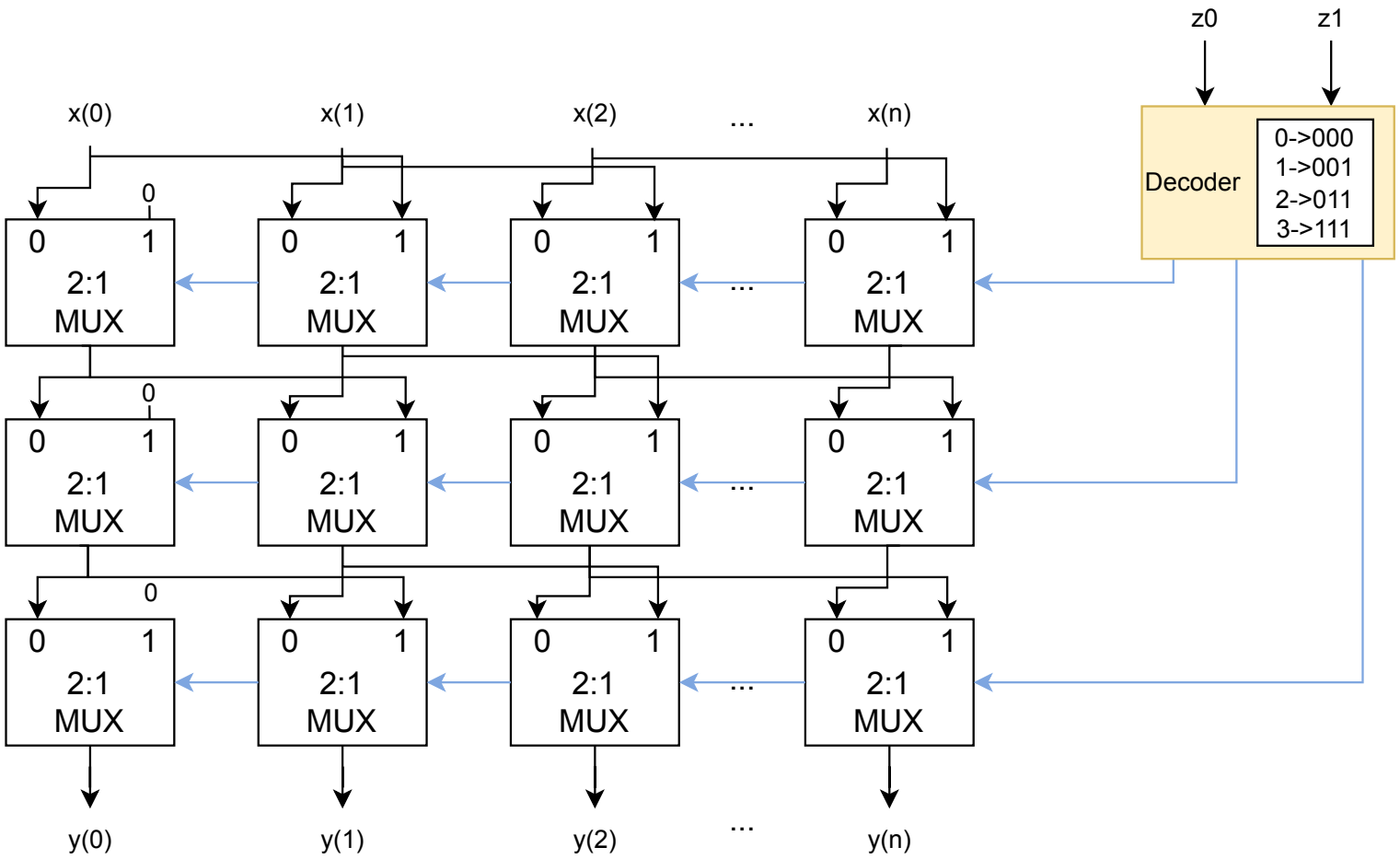
→ Die Bedingung in der `if generate`-Anweisung fragt den Wert des Parameters ab und erzeugt ggf. eine spezielle Instanziierung.

Iterative Instanziierung: Beispiel

```
library IEEE;
use IEEE.std_logic_1164.all;
entity ADDER is
  generic (BITS : integer);
  port ( X, Y   : in  std_ulogic_vector(BITS-1 downto 0);
         Res    : out std_ulogic_vector(BITS-1 downto 0));
end ADDER;
architecture struct of ADDER is
  component FA is
    port ( Op1, Op2, Ci : in  std_ulogic;
           Res, Co      : out std_ulogic);
  end component;
  constant LOW : std_ulogic := '0';
  signal carry : std_ulogic_vector(X'range);
begin
  RCA  : for i in 0 to BITS-1 generate
    GEN_FA_0 : if (i = 0) generate
      FA_0 : FA port map(X(i), Y(i), LOW,
                         Res(i), carry(i+1));
    end generate GEN_FA_0;
    GEN_FA_i : if (i > 0) generate
      FA_i : FA port map(X(i), Y(i), carry(i),
                         Res(i), carry(i+1));
    end generate GEN_FA_i;
  end generate RCA;
end architecture struct;
```

Strukturelle Beschreibung

Was macht folgende Schaltung?



Strukturelle Beschreibung

Implementieren Sie die Schaltung in VHDL mithilfe von for-generate-Loops.